

复旦大学课程项目报告

復旦大學

A Rhythm Game Architecture on FPGA

小组成员：

陈奕宽 25213040003

陈之逸 25213040070

林炫达 24212020013

孟思思 24212020144

薛文宵 24212020213

2025 年 12 月 14 日

目录

1	简介	3
1.1	项目背景与实验要求	3
1.2	系统整体设计	3
1.3	项目分工与协作模式	4
1.4	工具链	4
2	整体框架	4
3	软件设计	6
3.1	软件界面展示	6
3.2	普通模式和随机模式流程	6
3.2.1	普通模式 (Normal Mode)	6
3.2.2	随机模式 (Random Mode)	8
3.3	输入谱面格式	9
3.4	谱面引擎模块 (Chart Engine)	9
3.4.1	谱面校验函数 (chart_check)	9
3.4.2	随机谱面生成函数 (generate_random_chart)	12
3.4.3	ROM 生成函数 (process_chart)	14
3.5	谱面可视化分析模块 (Chart Analysis)	17
3.5.1	系统架构设计	17
3.5.2	核心实现流程	18
3.5.3	技术实现细节	19
3.5.4	使用方式与前端集成	21
3.5.5	功能总结与扩展方向	21
3.6	音乐播放模块 (Music Sync)	22
3.6.1	监听键盘及音乐播放	22
3.6.2	软硬件协同方案探索	22
4	硬件设计	24
4.1	顶层模块	24
4.2	分布式架构设计	24
4.3	可调节时钟分频器	25
4.4	按键消抖	25
4.5	随机数生成	26
4.6	TextLCD 显示系统	26
4.6.1	队列设计	26
4.6.2	LCD 控制逻辑	27

4.6.3	双频率设计	28
4.7	七段数码管显示	28
4.8	判定逻辑	28
4.8.1	硬件判定标准	28
4.8.2	判定信号锁定 (Signal Judged)	29
4.8.3	有限状态机设计	31
4.8.4	Vivado 行为级仿真	31
4.9	分数累加	33
4.9.1	核心逻辑与层次化构建	33
4.9.2	预加法优化	34
5	讨论	34
5.1	设计技巧	34
5.2	信号穿透	34
5.3	软硬件不一致性	34
5.4	Quartus 调试	34
6	总结	35
7	未来展望	35

1 简介

1.1 项目背景与实验要求

在嵌入式系统设计课程实验提供的三个选题方向（CPU 扩展、VGA 游戏、Web 控制）中，我们选择了题目二：基于 FPGA 的 VGA 游戏设计与改进。

基于题目要求，本项目不仅在 Altera DE2-115 FPGA 开发板上复刻了游戏 *Muse Dash* 的核心玩法，更旨在打造一个接近 Minimum Viable Product (MVP) 的基础音乐游戏 FPGA 工程化框架。为了突破传统硬件开发的局限性，我们构建了一套高度自动化的软硬件协同开发流程，实现了以下核心技术成果：

- **硬件可调与解耦：**构建以 Python 为核心的软件后端，将游戏参数与配置从 Verilog 代码中抽离。通过接口驱动硬件行为，避免了频繁修改 HDL 代码带来的繁琐流程。
- **模式抽象与多变：**对谱面数据与策略逻辑进行了深度抽象，支持多种生成路径（如固定谱面、随机生成）并可灵活切换，增强了系统的扩展性。
- **用户友好系统：**引入 Web 前端系统，实现前端-后端-硬件的紧密协同与实时反馈。这不仅实现了紧密的实时反馈，还增加了选曲、预览等贴合实际游戏场景的交互设计，极大地提升了系统的用户友好度。
- **软硬协同探索：**在实现基础逻辑的同时，进一步探索了 PC 音频播放与 FPGA 逻辑运行的同步机制。

综上所述，本项目打通了从谱面解析、算法处理、自动生成 ROM 代码到板级验证的完整闭环，是一次从单纯的硬件逻辑设计向软硬件深度融合的系统级设计跨越。

1.2 系统整体设计

本项目复刻了知名节奏游戏 **Muse Dash**（喵斯快跑）的核心玩法。

- **游戏机制：**系统在 FPGA 端复刻了经典的双轨道（上层/下层）判定机制。玩家需要根据背景音乐的节奏，通过按键击打对应轨道上飞驰而来的敌人。
- **软硬协同架构：**
 - **软件端（上位机）：**负责复杂的离线数据处理，包括音频播放、谱面文件的解析、特征提取、可视化分析、难度评估以及 Verilog ROM 文件的自动化生成。
 - **硬件端（FPGA）：**负责毫秒级的实时交互响应，包括 TextLCD 的动态滚屏显示、七段数码管的实时计分反馈以及音频同步信号的触发。



图 1: Muse Dash

1.3 项目分工与协作模式

本项目采用了现代软件工程中的模块化与接口隔离设计模式，团队成员通过 Git 进行协作，通过标准化的 JSON 协议与 API 接口实现模块解耦。具体分工如下：

1.4 工具链

- 版本控制：Github
- Verilog 编程：VSCode + Vivado
- Python 编程：VSCode
- 行为级仿真：Vivado
- 硬件验证：DE2-115 开发板 + Quartus 平台

2 整体框架

本项目构建了一个完整的软硬协同系统，其整体架构涵盖了用户交互层、后端处理层以及 FPGA 硬件层，通过 Web 前端、Python 后端服务器与 Quartus Prime 工具链实现了全流程的数据流转与控制。

- 陈之逸
- 软件框架设计：设计上位机交互逻辑，实现模式切换、Quartus 自动化调用及音频预览功能，完成系统框架搭建。
 - Verilog 编程与验证：负责顶层模块的编写和全系统的上板校验。
- 陈奕宽
- Verilog 编程与验证：负责核心硬件模块的 RTL 设计与仿真。
 - 报告撰写与演示：负责 LaTeX 实验报告的撰写，并制作答辩演示文稿并负责项目最终展示。
- 林炫达
- 谱面可视化分析：实现 chart_analysis 模块，开发谱面内容的深度统计逻辑与可视化绘图引擎。
- 薛文宵
- 谱面核心引擎：负责 chart_engine 模块，实现谱面校验、BPM 更新及随机谱面生成。
- 孟思思
- 音乐播放及软硬协同方案探索：负责编写基于空格键的音乐播放代码，并探索了软硬协同播放音乐的多种方式。

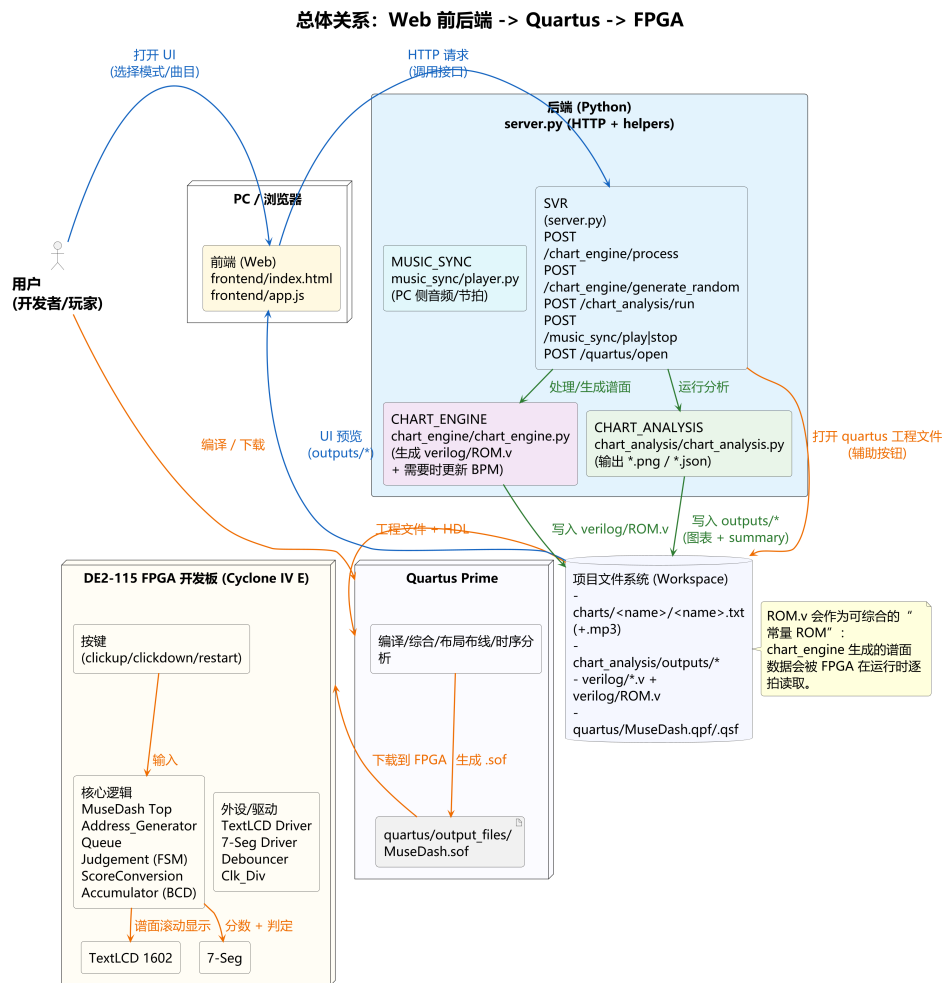


图 2: 整体框架

- **前端 (Web):** 位于 PC 浏览器端, 由 `frontend/index.html` 和 `frontend/app.js` 组成。它不仅是用户 (开发者/玩家) 的交互入口, 用于打开 UI 界面并选择游戏模式或曲目, 还负责向后端发送 HTTP 请求以调用各项功能接口, 并实时预览生成的图表和分析结果。
- **后端 (Python):** 核心是 `server.py`, 集成了多个功能模块:
 - **SVR (Server):** 接收前端的 POST 请求 (如谱面分析、随机生成、播放控制等), 并分发给相应的处理引擎。
 - **CHART_ENGINE:** 负责核心谱面处理, 包括生成 Verilog 可综合的 ROM 代码 (`verilog/ROM.v`) 以及在需要时更新 BPM 参数。
 - **CHART_ANALYSIS:** 执行谱面数据的深度分析, 并将结果输出为 PNG 图片和 JSON 数据, 供前端调用展示。
 - **MUSIC_SYNC:** 控制 PC 侧的音频播放与节拍同步。
- **Quartus Prime 与 FPGA:** Quartus Prime 负责对项目工程文件进行编译、综合、布局布线及时序分析, 最终生成 `.sof` 配置文件并下载到 DE2-115 FPGA 开发板。FPGA 硬件端包含了核心逻辑 (如地址生成、队列管理、判定状态机、分数累加等) 及外设驱动 (TextLCD 1602 显示谱面、7-Seg 数码管显示分数与判定), 通过读取后端生成的 `ROM.v` 数据实现游戏运行。

3 软件设计

3.1 软件界面展示

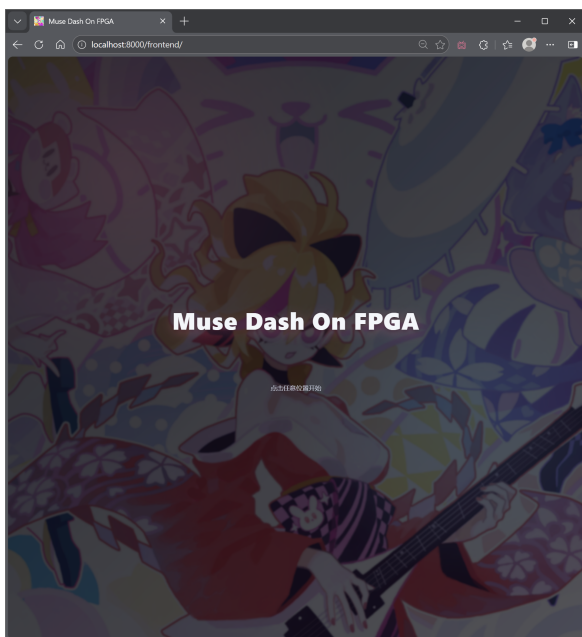
本实验的上位机界面采用 Web 前端技术构建, 旨在提供直观、友好的交互体验。如图 3 所示, 用户界面的交互逻辑严格遵循线性引导流程: 系统启动后首先进入欢迎页 (图 3a); 随后跳转至模式选择菜单 (图 3b), 用户可在此决定游戏方式。若选择普通模式 (图 3c), 界面将展示曲目列表供用户挑选; 若选择随机模式 (图 3d), 则会自动运行代码生成随机谱面。

3.2 普通模式和随机模式流程

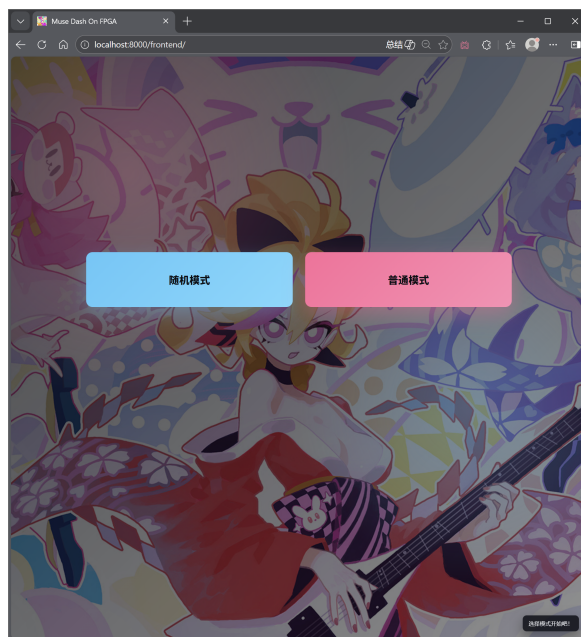
系统设计了两种主要的工作模式, 分别对应不同的数据处理与交互逻辑, 通过 Web 前端与 Python 后端的紧密配合实现。

3.2.1 普通模式 (Normal Mode)

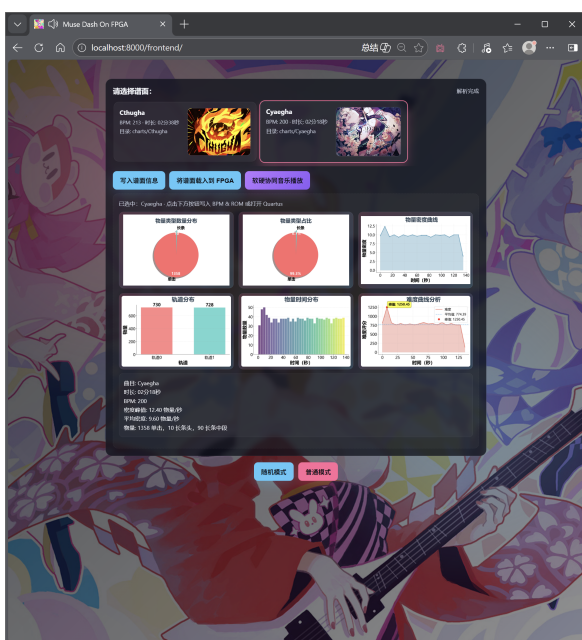
在普通模式下, 用户选择既有的曲目进行分析与游戏, 具体数据流转流程如下:



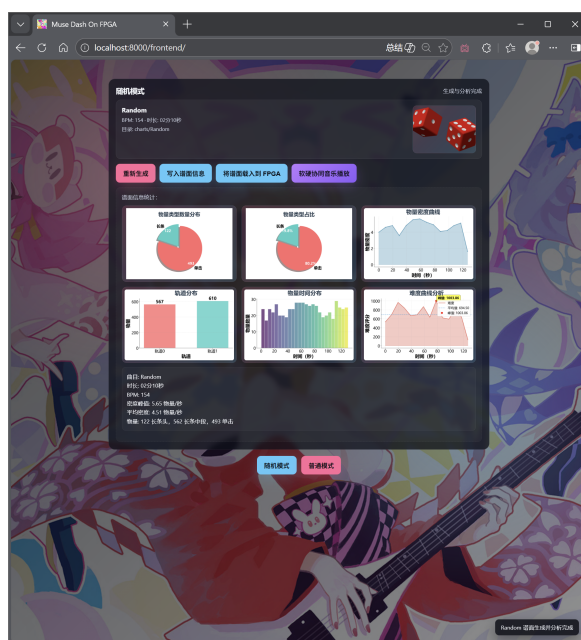
(a) 系统启动界面



(b) 模式选择菜单



(c) 分支一：普通模式选中



(d) 分支二：随机模式选中

图 3: 上位机 UI 交互流程展示：从启动到模式选择

1. **分析请求**: 前端向后端发送 POST /chart_analysis/run 请求。后端 ANALYSIS 模块读取指定谱面并执行分析，将结果（图表与 JSON）写入 /outputs/ 目录。
2. **结果获取**: 前端通过 GET 请求读取输出目录中的分析结果并在 UI 上进行展示。
3. **谱面处理**: 前端发起 POST /chart_engine/process 请求。后端 ENGINE 模块处理谱面数据，生成对应的 Verilog ROM 文件 (ROM.v)，为硬件综合做准备。
4. **播放启动**: 最后，前端发送 POST /music_sync/play 请求，后端 PLAYER 模块启动音频播放，标志着游戏的开始。

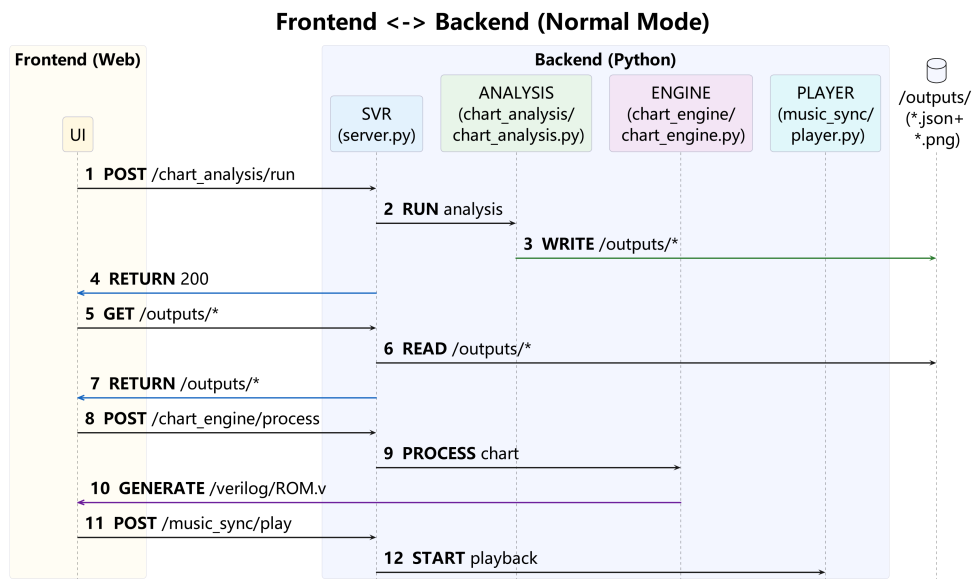


图 4: 普通模式前端流程图

3.2.2 随机模式 (Random Mode)

随机模式允许系统自动生成全新的谱面，流程包含额外的生成步骤：

1. **生成请求**: 前端首先发送 POST /chart_engine/generate_random 请求。后端 ENGINE 模块根据算法生成一个新的随机谱面 (RandomChart)。
2. **自动分析**: 谱面生成后，前端自动触发 POST /chart_analysis/run 请求。后端对新生成的随机谱面进行分析，并将结果写入 /outputs/。
3. **结果展示**: 前端通过 GET /outputs/Random_* 获取随机谱面的分析图表与数据进行预览。
4. **处理与播放**: 与普通模式类似，前端随后依次请求 PROCESS chart 生成 ROM 文件，并调用 music_sync/play 启动随机谱面的音频播放。

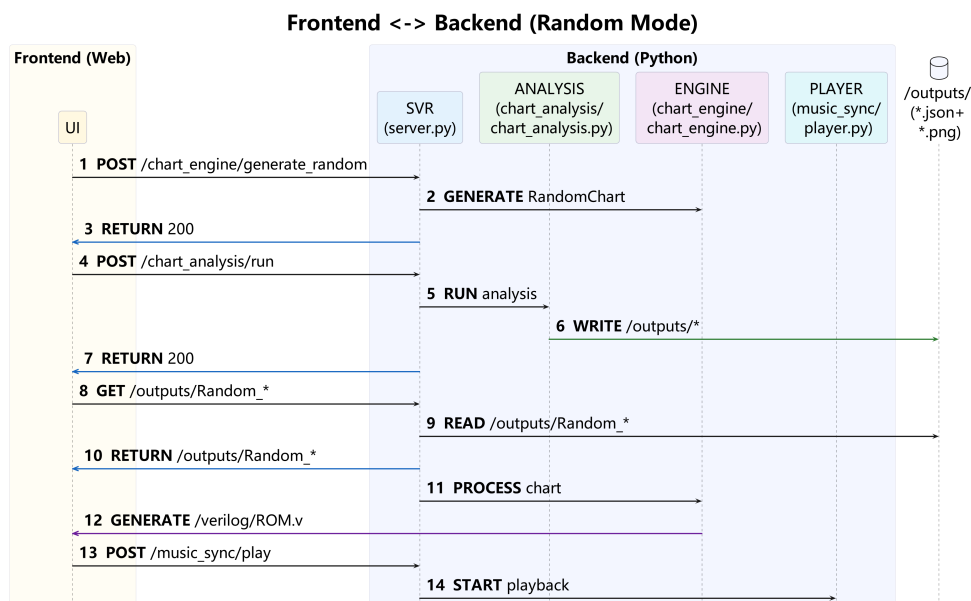


图 5: 随机模式前端流程图

3.3 输入谱面格式

我们设计了一种标准化的 txt 谱面格式。

参数 包含 BPM（曲速）以及每个音符的三个属性：**时间 (time)**、**类型 (type)**、**轨道 (track)**。

音符定义

- **Time:** 音符被判定的时钟周期数。
- **Type:** TAP (短按), HOLD_START (长按开始), HOLD_MIDDLE (长按保持)。
- **Track:** 上轨道 (1) 或下轨道 (0)。

3.4 谱面引擎模块 (Chart Engine)

本章节详细阐述了谱面引擎的三个核心组件：谱面校验、随机生成以及 ROM 编译。

3.4.1 谱面校验函数 (chart_check)

功能概述 谱面校验函数是谱面引擎的基础组件，负责对谱面文件进行全面的格式检查和逻辑验证，确保谱面文件符合游戏引擎的要求。该函数在谱面生成、ROM 编译等关键流程中起到质量保障作用，能够有效防止因格式错误导致的系统异常。实现采用早期失败策略，一旦发现错误立即返回并输出详细的错误信息，包括错误类型、出错行号和具体错误值，这大大提高了问题定位的效率。

实现思路 谱面文件采用简洁的文本格式：

- 第一行为 BPM 值声明，格式为 `bpm=< 整数 >`。
- 后续行为事件行，格式为 `(time,type,trace)`。其中 `time` 表示事件发生的时间点（非负整数），`type` 表示事件类型（`tap`、`hold_start`、`hold_mid`），`trace` 表示轨道编号（0 或 1）。

校验过程分为四个主要阶段：文件读取与基础检查、BPM 格式校验、事件行逐行校验和长条闭合检查。

在文件读取阶段，函数首先验证文件存在性和非空性。随后检查第一行的 BPM 格式，确保以 `bpm=` 开头且值为正整数。对于事件行的校验，函数维护两个关键状态：全局上一个事件的时间值 `last_time` 和每个轨道上一个事件的信息 `last_by_trace`。对每个事件行，函数首先使用正则表达式进行格式匹配，然后依次验证：

1. 事件类型有效性；
2. 轨道编号有效性；
3. 时间值非负性；
4. 全局时间单调性（允许不同轨道在同一时间点有事件，但整体时间必须单调不减）；
5. 同轨时间严格递增（同一轨道上的事件时间必须严格递增，不允许相等）。

长条事件的校验是函数的核心难点。要求 `hold_mid` 必须紧接在 `hold_start` 或 `hold_mid` 之后，且时间差必须为 1，这意味着长条事件必须是连续的时间序列。同时，`hold_start` 后必须跟随连续的 `hold_mid`，直到长条结束，不能出现 `hold_start` 后直接跟其他类型事件的情况。在遍历结束后，还会检查所有轨道是否有未闭合的 `hold_start`，确保每个长条事件都被正确闭合。

代码结构说明 核心函数是 `chart_check(chart_name, chart_path=None)`，接受曲目名称和可选的谱面文件路径作为参数，返回布尔值表示校验是否通过。

- 函数内部首先调用辅助函数 `_resolve_chart_path` 解析谱面文件路径，如果未提供路径则使用默认路径 `charts/<chart_name>/<chart_name>.txt`。
- 定义了三个关键常量：`_EVENT_PATTERN` 是用于匹配事件行格式的正则表达式，`_ALLOWED_TYPES` 定义了允许的事件类型集合，`_ALLOWED_TRACES` 定义了允许的轨道编号集合。

校验过程采用单次遍历的方式，时间复杂度为 $O(n)$ ，其中 n 为事件行数。实现使用正则表达式进行格式匹配，效率较高，内存占用小，仅维护必要的状态信息。这种设计使得函数具有良好的性能表现，即使对于包含大量事件的谱面文件也能快速完成校验。

简单示例 一个有效的谱面示例如下所示。该谱面包含 BPM 声明、单点事件和长条事件，所有事件都符合格式要求和逻辑约束。单点事件 (0,tap,0) 和 (5,tap,1) 分别位于不同轨道，时间单调递增。长条事件从时间 10 开始，包含 hold_start 和三个连续的 hold_mid，时间差均为 1，符合长条事件的连续性要求。

```
bpm=120
(0,tap,0)
(5,tap,1)
(10,hold_start,0)
(11,hold_mid,0)
(12,hold_mid,0)
(13,hold_mid,0)
(15,tap,1)
```

常见的错误情况包括时间不单调（如第二行时间值小于第一行）、同轨时间不严格递增（如同一轨道上出现相同时间的事件）、hold_mid 前没有 hold_start（长条事件必须以 hold_start 开始）以及 hold_start 未闭合（长条事件必须以 hold_mid 结尾）。函数能够准确识别这些错误并输出相应的错误信息。

工作流程 校验流程如下：

1. 从解析文件路径开始，首先检查文件是否存在，如果不存在则返回错误。
2. 文件存在后读取文件内容，检查文件是否为空，如果为空则返回错误。
3. 检查第一行的 BPM 格式，如果格式不正确则返回错误。
4. BPM 格式正确后，初始化校验状态，包括全局时间记录和每个轨道的事件记录。
5. 进入事件行遍历循环：
 - 对每个事件行首先进行格式匹配，如果格式不正确则返回错误。
 - 格式正确后提取时间、类型和轨道信息，依次校验事件类型有效性、轨道编号有效性、时间值非负性、全局时间单调性和同轨时间严格递增。
 - 对于 hold_mid 事件，额外检查是否紧接在前一个同轨的 hold_start 或 hold_mid 之后且时间差为 1。
 - 对于非 hold_mid 事件，如果前一个同轨事件是 hold_start，则必须当前事件是 hold_mid，否则返回错误。
 - 每个事件校验通过后更新状态记录，继续处理下一个事件。
6. 所有事件处理完成后，检查是否有未闭合的 hold_start，如果有则返回错误，否则返回校验通过。

3.4.2 随机谱面生成函数 (generate_random_chart)

功能概述 随机谱面生成函数实现了基于概率的随机谱面生成功能，能够根据指定的 BPM、时长、随机种子等参数自动生成符合格式要求的谱面文件。该函数采用基于节拍的循环生成策略，通过概率控制事件类型分布，生成包含单点事件和长条事件的随机谱面。支持随机种子功能，确保相同种子能够生成完全相同的谱面，这对于测试和调试具有重要意义。生成的谱面文件可以直接用于后续的校验和 ROM 编译流程。

实现思路 核心思想是在节拍序列上随机生成事件，使用概率控制事件类型的分布。首先根据 BPM 和时长计算总节拍数，公式为：

$$maxlength = \frac{bpm \times length_seconds}{60}$$

然后从节拍位置 0 开始，循环生成事件直到达到总节拍数。每次循环中，函数首先生成一个 2 到 6 之间的随机整数作为事件间隔，然后生成一个 1 到 100 之间的随机数用于决定事件类型，最后生成一个 1 到 3 之间的随机数用于决定轨道选择。

事件类型的选择基于概率分布：

- 当随机数小于等于 85 时生成单点事件 (tap)，概率为 85%；
- 当随机数大于 85 时生成长条事件 (hold)，概率为 15%。

这种分布设计使得生成的谱面以单点事件为主，长条事件作为点缀，符合大多数音乐游戏的谱面特征。

轨道选择有三种情况：trace 为 1 时仅在轨道 0 生成事件，trace 为 2 时仅在轨道 1 生成事件，trace 为 3 时在双轨道同时生成事件，这增加了谱面的多样性和挑战性。

对于单点事件，函数根据轨道选择在相应位置生成 tap 事件，然后更新节拍位置为当前位置加上事件间隔。对于长条事件，首先生成一个 3 到 8 之间的随机数作为长条长度，然后生成两个额外的随机数用于决定是否在长条首尾添加额外的单点事件（这种设计增加了谱面的复杂度）。长条事件的生成采用循环方式，在循环的开始位置生成 hold_start，在中间位置生成 hold_mid，在结束位置生成最后一个 hold_mid。长条生成完成后，更新节拍位置为当前位置加上事件间隔再加上长条长度减一。

代码结构说明 核心函数是 generate_random_chart(output_dir, name="Random", bpm=120, length_seconds=60, seed=None)，接受输出目录、文件名、BPM、时长和随机种子作为参数，返回生成的文件路径或 None（表示失败）。

函数首先设置随机种子（如果提供），这确保了相同种子能够生成相同的谱面。然后创建输出文件，如果目录不存在则自动创建。接下来计算总节拍数并初始化事件列表，列表的第一项为 BPM 声明行。

生成过程采用 while 循环，循环条件为当前节拍位置小于总节拍数。在循环内部，根据随机数决定生成单点事件还是长条事件，然后根据轨道选择生成相应的事件行并添

加到事件列表中。所有事件生成完成后，将事件列表写入文件。实现还包含一个测试函数 `test_generate_random_chart()`，其中包括基本功能测试、固定种子可重复性测试、文件格式验证、事件格式验证和错误处理测试。

错误处理机制完善，对于文件创建失败和文件写入失败都进行了异常捕获，并输出详细的错误信息。时间复杂度为 $O(n)$ ，其中 n 为总节拍数，空间复杂度也为 $O(n)$ ，需要存储所有事件行。由于使用 Python 标准库的 `random` 模块，随机数生成性能良好，整体性能表现优秀。

简单示例 假设使用参数 `bpm=120`、`length_seconds=60`、`seed=42` 生成谱面，可能得到如下输出。该谱面展示了单点事件和长条事件的混合分布，单点事件占主导地位，长条事件作为点缀。可以看到，单点事件的时间间隔在 2 到 6 个节拍之间随机变化，长条事件从时间 18 开始，包含 `hold_start` 和三个连续的 `hold_mid`，符合长条事件的连续性要求。双轨道同时触发的情况（如时间 12 处）增加了谱面的挑战性。

```
bpm=120
(3,tap,0)
(8,tap,1)
(12,tap,0)
(12,tap,1)
(18,hold_start,0)
(19,hold_mid,0)
(20,hold_mid,0)
(21,hold_mid,0)
(25,tap,1)
(30,tap,0)
```

工作流程 生成流程如下：

1. 从设置随机种子开始（如果提供），然后创建输出文件，如果文件创建失败则返回错误。
2. 文件创建成功后，计算总节拍数并初始化事件列表和节拍位置。
3. 进入主循环，循环条件为当前节拍位置小于总节拍数。
4. 在循环内部，首先生成事件间隔、事件类型随机数和轨道选择随机数。根据事件类型随机数判断生成单点事件还是长条事件。
5. 如果是单点事件，根据轨道选择生成相应的 `tap` 事件并添加到事件列表，然后更新节拍位置。

6. 如果是长条事件，首先生成长条长度和首尾额外事件标志，然后进入长条生成循环。在长条生成循环中，根据当前位置生成 `hold_start`（开始位置）、`hold_mid`（中间位置）或最后一个 `hold_mid`（结束位置），并根据首尾标志决定是否添加额外的 `tap` 事件。长条生成完成后，更新节拍位置并返回主循环。
7. 主循环结束后，将所有事件写入文件，如果写入失败则返回错误，否则返回生成的文件路径。

3.4.3 ROM 生成函数 (`process_chart`)

功能概述 ROM 生成函数实现了从谱面文件到 Verilog ROM 模块的完整转换功能，是连接软件谱面设计和硬件实现的关键桥梁。该函数读取经过校验的谱面文件，解析其中的事件信息，将其转换为 FPGA 可用的 ROM 数据格式，并自动生成标准的 Verilog ROM 模块代码。同时，会提取谱面文件中的 BPM 值，计算相应的时钟分频系数，并自动更新顶层 Verilog 模块的 BPM 参数，确保硬件时钟与谱面节拍同步。生成的 ROM 模块可以直接用于 FPGA 综合和实现，大大简化了从谱面设计到硬件部署的流程。

实现思路 核心任务是将文本格式的谱面文件转换为硬件可用的 ROM 数据。谱面文件中的每个事件包含时间、类型和轨道三个信息，需要将这些信息编码为 4 位的 ROM 数据。编码规则如下：

- 每个时间点的 ROM 数据为 4 位，低 2 位表示轨道 0 (`notedown`) 的事件类型，高 2 位表示轨道 1 (`noteup`) 的事件类型。
- 事件类型的编码映射为：无事件编码为 00，`tap` 编码为 01，`hold_start` 编码为 10，`hold_mid` 编码为 11。

这种编码方式使得同一时间点可以同时存在不同轨道的事件，通过位掩码操作可以独立更新每个轨道的数据。

BPM 到时钟分频的转换是该函数的另一个重要功能。FPGA 使用时钟分频来匹配 BPM，计算公式为：

$$div_cnt = \frac{375,000,000}{bpm}$$

其中 375,000,000 是系统时钟频率（假设为 375 MHz）。实现会提取谱面文件第一行的 BPM 值，计算相应的分频系数，然后使用正则表达式在顶层 Verilog 模块中查找并替换 `div_cnt` 参数的值，实现 BPM 的自动同步。

ROM 长度的计算需要考虑硬件实现的约束。ROM 长度必须是 2 的幂次，以便于地址解码，同时要能覆盖谱面中的最大时间值。实现会遍历所有事件找出最大时间值，然后计算大于等于该值的最小 2 的幂次作为 ROM 长度。由于硬件地址位宽限制为 12 位，ROM 长度最大为 4096，如果计算出的长度超过此限制，则返回错误。

代码结构说明 核心函数是 `process_chart(chart_name)`，接受曲目名称作为参数，返回布尔值表示处理是否成功。

函数首先解析谱面文件路径，检查文件是否存在，如果不存在则返回错误。然后调用 `chart_check` 进行谱面校验，如果校验失败则直接返回。校验通过后，读取谱面文件内容，解析第一行获取 BPM 值，计算时钟分频系数，并使用正则表达式更新顶层 Verilog 模块的 `div_cnt` 参数。

接下来，函数使用正则表达式解析所有事件行，提取时间、类型和轨道信息，同时记录最大时间值。根据最大时间值计算 ROM 长度，检查是否超过限制。然后初始化 ROM 数组，数组长度为计算出的 ROM 长度，所有元素初始化为 0。遍历所有事件，根据事件类型获取编码值，根据轨道选择更新 ROM 数组的相应位置。

- 对于轨道 1 的事件，使用位掩码操作更新高 2 位，保留低 2 位；
- 对于轨道 0 的事件，更新低 2 位，保留高 2 位。

这种设计确保了不同轨道的事件可以共存于同一时间点。

ROM 数据填充完成后，函数生成 Verilog 代码。代码包括模块声明、ROM 数组声明、初始化块和组合逻辑块。初始化块中包含所有 ROM 位置的初始化语句，使用二进制格式表示。组合逻辑块中将 ROM 数据按位域分解为 `noteup` 和 `notedown` 两个输出。生成的代码写入 `verilog/ROM.v` 文件，如果写入失败则返回错误。

简单示例 输入谱面文件包含 BPM 声明和多个事件，每个事件包含时间、类型和轨道信息。函数首先提取 BPM 值 120，计算分频系数为 3,125,000，并更新顶层模块的 `div_cnt` 参数。然后解析事件，找出最大时间值为 15，计算 ROM 长度为 16（2 的 4 次方）。

ROM 数据填充过程如下：

- 时间 0 的轨道 0 事件 `tap` 编码为 01，写入 ROM[0] 的低 2 位；
- 时间 5 的轨道 1 事件 `tap` 编码为 01，写入 ROM[5] 的高 2 位；
- 时间 10 的轨道 0 事件 `hold_start` 编码为 10，写入 ROM[10] 的低 2 位；
- 后续的 `hold_mid` 事件编码为 11，写入相应位置的低 2 位。

输入谱面示例：

```
bpm=120
(0,tap,0)
(5,tap,1)
(10,hold_start,0)
(11,hold_mid,0)
```

```
(12,hold_mid,0)
(15,tap,1)
```

生成的 Verilog ROM 模块包含模块声明、ROM 数组声明和初始化数据。ROM 数组大小为 16，每个位置存储 4 位数据。初始化块中为每个 ROM 位置赋值，使用 4 位二进制格式。组合逻辑块中将 ROM 数据按位域分解，低 2 位输出为 `notedown`，高 2 位输出为 `noteup`。硬件可以通过地址输入访问 ROM，获取对应时间点的事件信息。

```
module ROM (
    input [11:0] addr,
    output reg [1:0] noteup,
    output reg [1:0] notedown
);
reg [3:0] ROM [0:15];
initial begin
    ROM[0] = 4'b0001;
    ROM[5] = 4'b0100;
    ROM[10] = 4'b0010;
    ROM[11] = 4'b0011;
    ROM[12] = 4'b0011;
    ROM[15] = 4'b0100;
end
always @(*) begin
    {noteup, notedown} = ROM[addr];
end
endmodule
```

简要流程 处理流程如下：

1. 从解析谱面文件路径开始，检查文件是否存在，如果不存在则返回错误。
2. 文件存在后调用 `chart_check` 函数进行校验，如果校验失败则返回错误。
3. 校验通过后读取谱面文件内容，解析第一行获取 BPM 值，如果 BPM 格式不正确则返回错误。
4. BPM 格式正确后，计算时钟分频系数，使用正则表达式更新顶层 Verilog 模块的 `div_cnt` 参数，如果更新失败则返回错误。
5. 接下来解析所有事件行，提取时间、类型和轨道信息，同时记录最大时间值。

6. 根据最大时间值计算 ROM 长度，如果超过 4096 的限制则返回错误。ROM 长度确定后，初始化 ROM 数组，所有元素设为 0。
7. 遍历所有事件，根据事件类型获取编码值，根据轨道选择使用位掩码操作更新 ROM 数组的相应位置。对于轨道 1 的事件，更新高 2 位；对于轨道 0 的事件，更新低 2 位。
8. 所有事件处理完成后，生成 Verilog 代码。代码生成过程包括：添加模块声明和接口定义，添加 ROM 数组声明，添加初始化块并为每个 ROM 位置生成初始化语句，添加组合逻辑块实现数据输出。
9. 生成的代码写入 `verilog/ROM.v` 文件，如果写入失败则返回错误，否则返回处理成功。

整个流程确保了从谱面文件到硬件 ROM 模块的完整转换，同时实现了 BPM 参数的自动同步。

3.5 谱面可视化分析模块 (Chart Analysis)

本模块是 MuseDash 项目的重要组成部分，旨在对游戏谱面进行全面的统计分析与可视化展示。该模块能够自动扫描项目中的谱面目录，解析谱面文件，提取关键信息，并生成多种类型的统计图表和数据摘要，为谱面设计、难度评估和数据分析提供有力支持。

3.5.1 系统架构设计

模块结构 谱面分析模块采用模块化设计，主要包含以下组件：

- **主程序** (`chart_analysis.py`): 负责整体流程控制。
- **依赖管理** (`requirements.txt`): 确保运行环境的一致性。
- **输出目录** (`outputs/`): 存储所有生成的分析结果，包括协议文件、统计摘要和可视化图表。

核心类设计 系统采用面向对象的设计思想，将功能划分为三个核心类：

1. **ChartParser (谱面解析器)**: 负责读取和解析谱面 TXT 文件，从文件第一行提取 BPM 信息，逐行解析音符事件，并计算谱面总时长。
2. **ChartAnalyzer (谱面分析器)**: 对解析后的数据进行多维度统计分析，包括总音符数统计、类型分布统计、密度曲线计算、轨道分布统计、时间分布数据提取以及难度曲线计算。

3. **ChartVisualizer (谱面可视化器)**: 负责将统计数据转化为直观的可视化图表, 生成六种不同类型的图表文件。

3.5.2 核心实现流程

1. 谱面解析流程 系统首先扫描 `charts/` 目录下的所有谱面子目录, 对每个谱面目录进行以下处理: 检查谱面 TXT 文件是否存在, 调用 `chart_check()` 函数进行格式校验, 使用 `ChartParser` 解析谱面文件。解析过程中, 系统读取文件第一行提取 BPM 值, 使用正则表达式逐行匹配音符事件格式 (时间, 类型, 轨道), 将所有音符记录到列表中, 并计算最大时间作为谱面时长。解析完成后, 系统将 BPM、音符列表和时长信息传递给分析器。

2. 统计分析流程 `ChartAnalyzer` 接收解析后的数据, 执行全面的统计分析:

- **音符计数**: 在总音符数统计方面, 系统只统计 `tap` 和 `hold_start` 类型的音符, 避免 `hold_mid` 音符的重复计算, 确保统计结果的准确性。
- **密度曲线计算**: 采用时间窗口方法, 将谱面时长划分为多个时间窗口, 窗口大小根据谱面总时长动态调整 (公式为 $\max(100, \text{duration}/100)$)。随后统计每个窗口内的音符数量, 生成时间-密度映射关系, 有效反映谱面在不同时间段的音符密度变化。
- **难度曲线计算**: 这是系统的核心功能之一。为了量化谱面的瞬时难度, 系统建立了一个综合考量音符类型、轨道并发数及密度的多因子加权模型。难度分数 $Score(t)$ 的计算公式如下:

$$Score(t) = \left(\sum w_{type} \right) \times [1 + 0.2 \times (N_{track} - 1)] \times [1 + 0.1 \times (N_{notes} - 1)] \quad (1)$$

其中各项参数定义为:

- $\sum w_{type}$ **基础权重和**: 窗口内所有音符权重的总和。具体权重设定为: Tap=1.0, Hold Start=1.5, Hold Mid=0.3, Hold End=0.5。
- N_{track} **轨道复杂度**: 当前时间窗口内活跃的轨道数量。当多轨并发时, 难度系数按线性增长 (每增加一轨权重增加 0.2)。
- N_{notes} **密度因子**: 当前窗口内的音符总数。音符越密集, 手速要求越高, 难度系数随之增长 (每增加一个音符权重增加 0.1)。

最终生成的难度曲线不仅反映了谱面的张力变化, 系统还会据此自动标记全曲的“最难点” (Peak Difficulty)。

3. 可视化生成流程 `ChartVisualizer` 根据分析结果生成六种不同类型的可视化图表, 所有图表均采用 200 DPI 的高分辨率输出:

- **音符类型数量饼图:** 展示各类型音符的绝对数量 (实际数字)。
- **音符类型占比饼图:** 展示各类型音符的相对比例 (百分比)。
- **密度曲线图:** 使用折线图和填充区域展示谱面随时间变化的音符密度。
- **轨道分布柱状图:** 展示每个轨道的音符数量, 并在柱状图上直接显示数值标签。
- **音符时间分布直方图:** 展示音符在时间轴上的分布情况, 根据谱面时长动态调整 bins 数量。
- **难度曲线分析图:** 综合考虑密度、音符类型复杂度和轨道分布, 显示平均难度线和峰值标记。

图表设计采用了现代配色方案 (红、青、蓝渐变色), 并优化了字体大小和粗细, 添加了阴影效果增强立体感, 曲线图和柱状图添加网格线以辅助数值读取。

4. 数据输出流程 系统生成两种主要的数据输出文件:

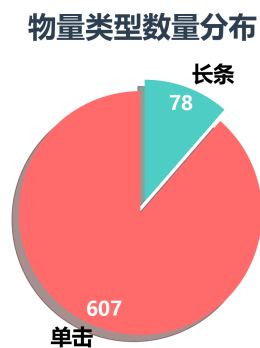
- **summary.json:** 包含谱面的核心统计信息, 包括曲目名、BPM 值、谱面时长、总音符数、Tap/Hold_start 音符数量、各类型音符数量分布、轨道分布、密度峰值和平均密度。
- **protocol.json:** 作为协议文件, 汇总所有谱面的信息, 包括每个谱面的名称、生成的图表文件列表、summary 文件路径, 以及可选的 BPM、时长、文件夹名和音频文件名。

3.5.3 技术实现细节

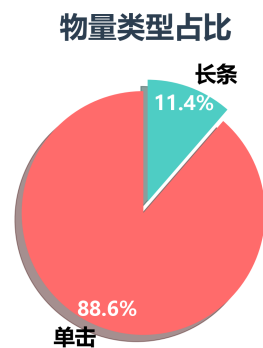
依赖库选择

- **matplotlib:** 作为主要的图表生成库, 支持多种图表类型, 输出质量高。
- **numpy:** 用于数值计算和颜色生成, 提供高效的数组操作和数学运算支持。
- **json/pathlib/re:** 分别用于 JSON 读写、跨平台路径处理及正则表达式解析。

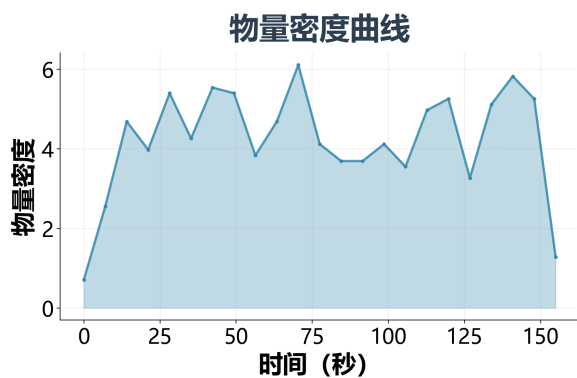
谱面文件格式解析 谱面文件采用文本格式, 第一行为 BPM 信息 (`bpm= 数值`), 后续每行表示一个音符事件 (时间, 类型, 轨道)。其中时间为整数, 类型包括 `tap`、`hold_start` 等, 轨道为 0 或 1。系统使用正则表达式精确匹配这种格式, 确保解析的准确性。



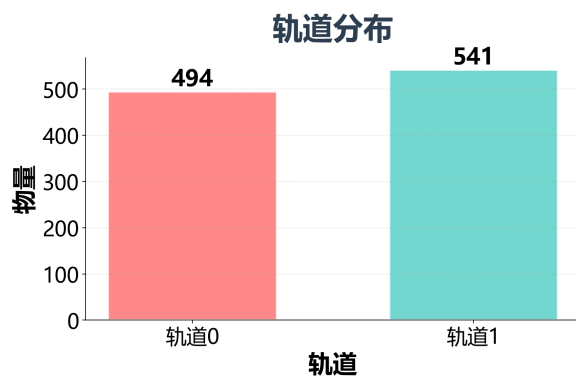
(a) 音符类型数量饼图



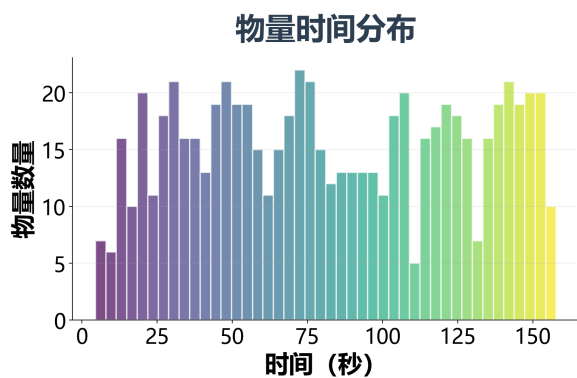
(b) 音符类型占比饼图



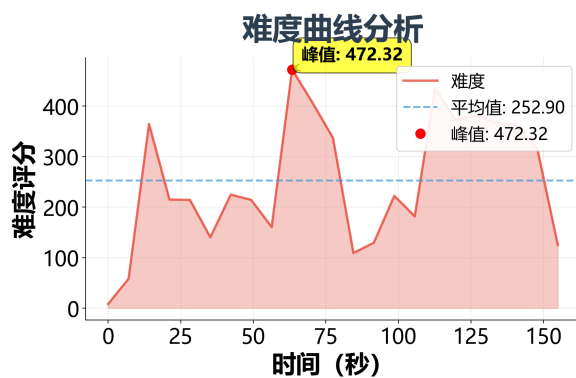
(c) 密度曲线图



(d) 轨道分布柱状图



(e) 音符时间分布直方图



(f) 难度曲线分析图

图 6: Cthugha 谱面数据可视化分析详情

图表尺寸适配 为了适配前端显示需求，系统对不同类型的图表采用不同的尺寸策略：

- **饼图**：使用 6×6 英寸的正方形尺寸，不使用 `bbox_inches='tight'` 以保持固定比例。
- **其他图表**（曲线、柱状、直方）：统一使用 9×6 英寸的 3:2 比例，与前端容器的 180×120 像素比例匹配，确保图表在前端完整显示而不被裁剪。

特殊处理机制 系统对 Random 谱面采用与其他谱面相同的处理逻辑，保证一致性。在统计总音符数时，特意剔除 `hold_mid` 音符以避免重复计数。同时，系统明确区分了 `note_count`（整数）和 `note_density`（百分比）两种饼图，满足不同分析需求。对于空数据情况，系统会生成空图表占位，避免程序异常。

3.5.4 使用方式与前端集成

环境配置与运行 使用前需执行 `pip install -r requirements.txt` 安装依赖（要求 Python 3.6+）。在目录下执行 `python chart_analysis.py` 即可自动扫描 `charts` 目录，处理完成后结果保存在 `outputs/` 中。

输出文件与前端集成 输出目录包含 `protocol.json` 协议文件、每个谱面的 `summary.json` 及六种 PNG 图表。前端系统通过读取 `protocol.json` 获取所有谱面信息列表，进而加载对应的图表（标准 PNG 格式，直接嵌入 HTML）和统计摘要（JSON 格式，便于 JavaScript 解析）。

3.5.5 功能总结与扩展方向

当前版本已实现完整的谱面分析功能，包括自动扫描解析、多维度统计（总数、类型、密度、轨道、时间、难度）、高质量可视化生成及协议文件输出。系统设计合理，功能完善。

未来扩展方向包括：

- 支持多 BPM 谱面并绘制 BPM 变化曲线。
- 使用 Plotly 生成可交互的 HTML 图表。
- 添加连击数、最大连击、平均间隔等更多统计指标。
- 生成热力图，展示音符在时间和轨道上的热力分布。

3.6 音乐播放模块 (Music Sync)

3.6.1 监听键盘及音乐播放

该模块实现了一个简单的“按空格播放音频”功能。当模块调用时，用户按下空格立即播放对应音乐，若指定路径中无同名音乐，则播放一段默认节拍。

模块顶部根据当前文件目录计算 `BASE_DIR`，并定位一个回退默认音频文件 `DEFAULT`。模块主接口是 `listen_and_play(chart_name)`，它接受一个曲目名或文件路径，若传入的是曲目名则按约定路径 `charts/< 曲目名 >/< 曲目名 >.mp3` 查找音频，找不到时回退到 `DEFAULT`。

初始化方面，`_init_pygame()` 用一个模块级布尔变量 `pygame_initied` 做“只初始化一次”的标志：首次调用时尝试运行 `pygame.mixer.init()`，成功则把标志设为 `True`，失败则捕获异常并打印错误信息。该实现避免了重复初始化。

音频播放由 `_play_async(path)` 完成：先检查路径存在性，然后启动一个守护线程执行加载并播放操作。线程内先调用 `_init_pygame()` 确保 `mixer` 已就绪，再用 `pygame.mixer.music.load/play()` 播放。使用守护线程意味着如果主线程很快退出，播放线程会被强制终止；同时 `pygame.mixer.music` 是单通道播放，快速连续触发会中断正在播放的音轨，并重新播放音频。

`listen_and_play(chart_name)` 的主循环使用 `keyboard.is_pressed("space")` 轮询检测空格按下事件，触发后调用 `_play_async` 并通过 `time.sleep(0.3)` 做简单去抖，避免长按重复触发，主循环每次迭代以 `0.01s sleep` 降低 CPU 占用。按 `Ctrl+C` 可以中断并退出监听。`main()` 为调试入口，可调用 `listen_and_play` 直接运行测试。

- 需要安装的 python 依赖库：`pygame/keyboard`
- MuseDash-main 目录下调用调试命令：`python music_sync/player.py songName`

3.6.2 软硬件协同方案探索

由于时间限制，本项目虽未能在硬件上完全部署该协同功能，但我们对多种软硬件交互技术路线进行了详尽的理论探索与对比分析，并最终甄选出一种在当前约束下的最优方案。以下是我们调研的四种主流技术路径：

- 方案一：PC 键盘 → 软件捕获 → 串口 (UART) → FPGA
 - 用户通过键盘在 PC 端按下空格键后，软件立即捕获该事件，并通过 UART 串口向 FPGA 发送标准化控制指令帧，例如包含播放启动标志位与 BPM 参数的数据包。FPGA 内部通过 UART 接收模块解析命令，并驱动 Address Generator 与节拍计数器从零时刻开始运行，从而实现音乐播放起始点与谱面滚动的同步。
 - 优点：软硬件接口简单、协议可定制、调试直观、时延可控

- 缺点：串口本身存在缓冲与时序抖动，时间精度一般，通常在毫秒级

- **方案二：USB HID 键盘直通 FPGA（硬件级 USB 输入）**

- 将键盘直接接入 FPGA，由 FPGA 作为 USB 主机对 USB HID 键盘进行枚举和轮询，从而绕过 PC 端的操作系统与软件栈，使按键事件直接驱动硬件状态机。结构为键盘 → FPGA USB Host 控制器 → HID 解码 → 播放/节拍控制 FSM。
- 优点：无串口依赖，时序稳定性好（USB HID 的轮询周期通常固定在 1ms 左右，事件响应路径更短）
- 缺点：需要在 FPGA 内实现完整的 USB Host 协议栈与 HID 协议解析模块，对逻辑资源和开发经验要求较高，不适合快速原型开发场景

- **方案三：PC 键盘 → 软件 → GPIO 脉冲 → FPGA（低延迟硬件触发）**

- 这是一种基于 GPIO 的硬件触发方式。该方案中，上位机软件在检测到键盘事件后，通过 USB-GPIO 转换模块或单片机输出一个标准 TTL 电平的短脉冲信号，该脉冲直接接入 FPGA 的通用 I/O 引脚。FPGA 在检测到该引脚的上升沿后立即清零内部计数器并启动播放状态机。
- 优点：延迟极低（省去串口协议解析过程），同步精准，比串口稳定，适合对实时性要求较高的音乐节奏类应用
- 缺点：需要额外的硬件接口电路，系统集成时要额外考虑电气兼容性与可靠性

- **方案四：USB HID 设备增加一条“同步通道”（高级软硬件协同）**

- 这是一种基于 USB 复合设备的同步架构。该方案通过在系统中构建一个同时具有标准 HID 键盘接口和自定义控制接口的 USB 设备，使 FPGA 能够通过 USB 通道接收按键数据和专用同步指令。
- 优点：接口统一、扩展性强，可以在同一物理链路上实现多通道控制和状态反馈，工业设备常用
- 缺点：需要在 FPGA 内实现 USB Device 协议栈及自定义 HID 报告解析，软硬件复杂度高，实现成本高

综合系统可靠性、实现难度与开发周期等因素，对于当前 MuseDash 系统而言，最适合采用“PC 键盘 → 上位机软件 → UART 串口 → FPGA”这一协同架构。该架构既可以满足基本的播放启动与节拍同步需求，也便于在后续阶段引入时间戳校准机制或硬件脉冲触发机制进行性能优化。通过在 FPGA 端加入简单的串口命令解析逻辑并与现有 Address Generator、Judgement FSM 和 Score Accumulator 模块联动，可以在不大幅增加系统复杂度的前提下，实现稳定可靠的软硬件协同控制效果。

4 硬件设计

4.1 顶层模块

顶层模块 (MuseDash.v) 是整个硬件系统的核心模块，其主要职责是把各个子模块实例化并连接起来，让它们能够协同工作。此外，顶层模块还承担了信号处理任务：一方面，顶层模块负责接收来自开发板的复位信号（包括低电平有效的全局复位 `rst_n` 和游戏重置信号 `restart`），并将这些控制信号同步分发至所有子模块。另一方面，因为不同模块间传输的音符和分数数据位宽不同，我们需要在顶层对这些信号线进行切片与拼接，确保数据能准确地流向该去的地方。这种设计让复杂的连线逻辑都集中在顶层，子模块只需要专注于自己的功能，让整个系统结构看起来更有条理。

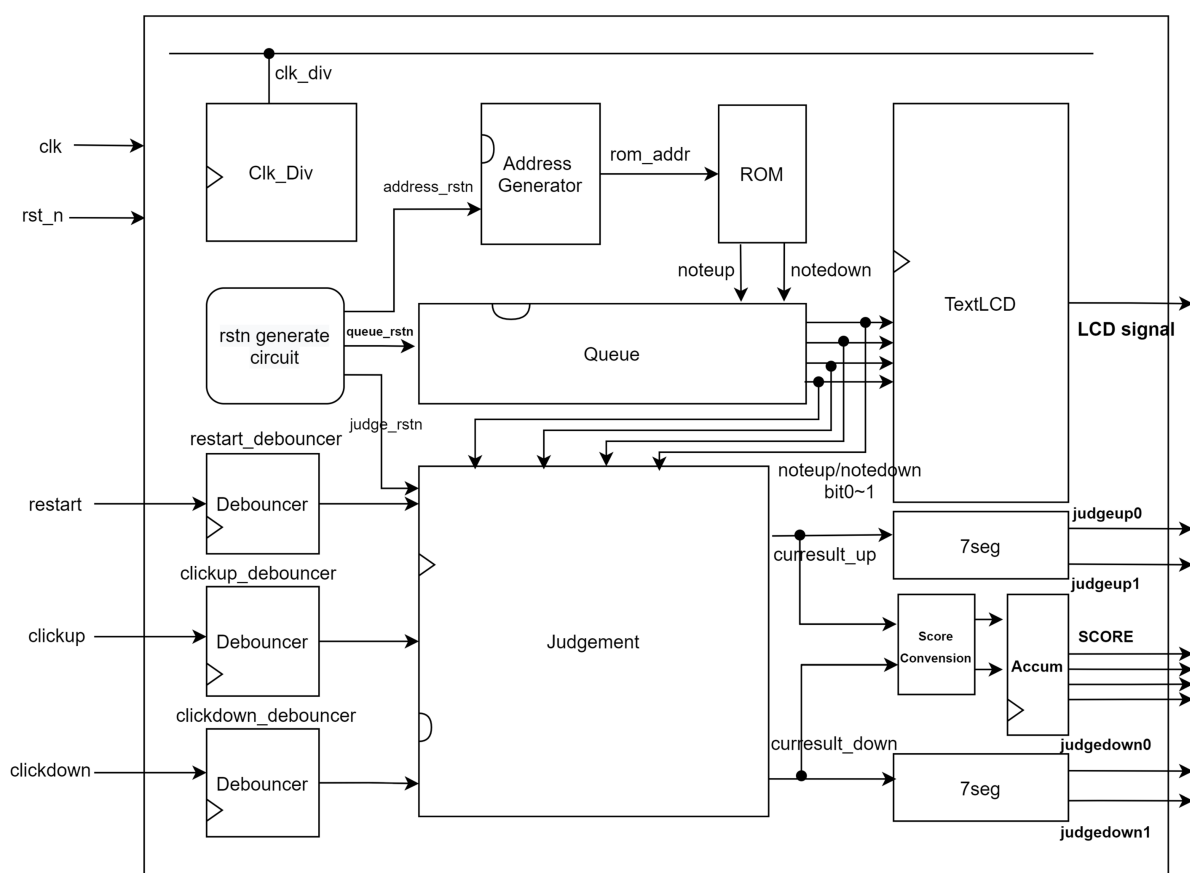


图 7: 顶层模块架构图

4.2 分布式架构设计

为了降低调试难度并提高团队协作效率，我们采用了分布式架构设计。整个设计被细分为 14 个轻量级模块，每个模块专注于单一功能。如图 7 所示，系统包含负责生成可变频率时钟的 `Clk_Div` 模块、负责按键消抖的 `Debouncer` 模块、以及用于随机数生成的 `LFSR` 模块。在显示方面，`CurrentJudge7Seg` 和 `TotalScore7Seg` 分别处理判定

结果和总分的数码管显示，而 TextLCD 则驱动液晶屏显示滚动的谱面。核心数据流由 Address_Generator 驱动 ROM 读取，经由 Queue 模块处理后进入 Judgement 模块进行判定。这种分布式的设计使得每个模块都相对独立且易于验证，在最终测试阶段，我们只需重点关注核心的判定逻辑，极大地缩短了开发周期。

4.3 可调节时钟分频器

Clk_Div 模块旨在根据输入的 50MHz 系统时钟 (clk) 生成一个分频后的低频时钟信号 (clk_div)。该模块引入了一个关键参数 div_cnt（分频系数）。默认情况下，该参数使得输出时钟周期为 100 毫秒（10Hz）。模块内部维护一个计数器，每当计数器达到 div_cnt 定义的阈值时，翻转输出电平并重置计数器。

该模块最显著的特性是其参数的可调节性。我们将 div_cnt 参数暴露给顶层模块，使得软件端的 Python 程序可以根据乐曲的 BPM（每分钟节拍数）动态计算出精确的分频值，并在生成 ROM 时自动更新顶层模块的参数。这一设计实现了软硬件的深度同步，确保游戏速度严格匹配音乐节奏。

```
module MuseDash #(
    parameter div_cnt = 2049180
    // div_cnt = 50,000,000 / (bpm * 4 / 60) / 2 = 375,000,000 / bpm;
) (
```

图 8: 顶层模块中的 div_cnt 参数定义

4.4 按键消抖

Debouncer 模块用于消除机械按键在闭合或断开瞬间产生的抖动噪声，确保输出信号 (click_out) 的稳定性。其工作原理基于计数器监测：当检测到输入信号变化且模块未处于计数状态时，启动一个约 20ms 的倒计时。在此期间，任何输入变化都会被忽略。只有当计数器归零且输入信号依然稳定时，输出信号才会更新。虽然这一机制引入了微小的延迟，但这对于滤除瞬态毛刺、保证 Judgement 模块接收到干净的脉冲信号至关重要。

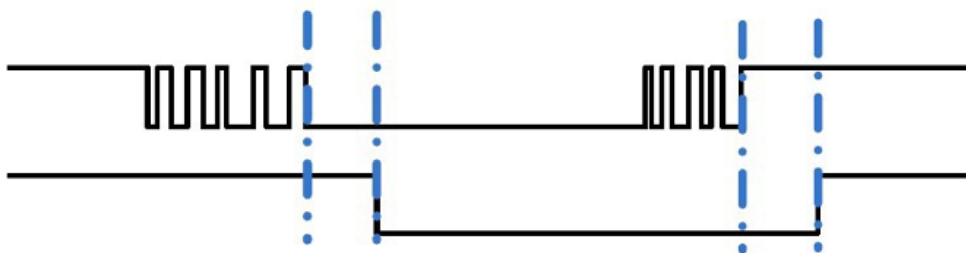


图 9: 消抖模块原理

4.5 随机数生成

为了在代码未完全完成时进行随机输入测试,我们利用线性反馈移位寄存器 (LFSR) 实现了伪随机序列生成 (PRBS)。该电路由 n 个触发器和若干异或门组成。在每个时钟周期,新的输入值通过对寄存器特定位置的异或运算反馈生成。我们采用 13 位 LFSR, 根据特定的反馈多项式 (见图 10), 能够产生最大重复间隔为 $2^{13} - 1$ 的伪随机序列。该模块为验证队列移位和判定逻辑的鲁棒性提供了自动化的测试激励。

Bits (n)	Feedback polynomial	Taps	Taps (hex)	Period ($2^n - 1$)
2	$x^2 + x + 1$	11	0x3	3
3	$x^3 + x^2 + 1$	110	0x6	7
4	$x^4 + x^3 + 1$	1100	0xC	15
5	$x^5 + x^3 + 1$	10100	0x14	31
6	$x^6 + x^5 + 1$	110000	0x30	63
7	$x^7 + x^6 + 1$	1100000	0x60	127
8	$x^8 + x^6 + x^5 + x^4 + 1$	10111000	0xB8	255
9	$x^9 + x^5 + 1$	100010000	0x110	511
10	$x^{10} + x^7 + 1$	1001000000	0x240	1,023
11	$x^{11} + x^9 + 1$	10100000000	0x500	2,047
12	$x^{12} + x^{11} + x^{10} + x^4 + 1$	111000001000	0xE08	4,095
13	$x^{13} + x^{12} + x^{11} + x^8 + 1$	1110010000000	0x1C80	8,191
14	$x^{14} + x^{13} + x^{12} + x^2 + 1$	11100000000010	0x3802	16,383
15	$x^{15} + x^{14} + 1$	11000000000000	0x6000	32,767
16	$x^{16} + x^{15} + x^{13} + x^4 + 1$	1101000000001000	0xD008	65,535
17	$x^{17} + x^{14} + 1$	1001000000000000	0x12000	131,071
18	$x^{18} + x^{11} + 1$	10000001000000000	0x20400	262,143
19	$x^{19} + x^{18} + x^{17} + x^{14} + 1$	111001000000000000	0x72000	524,287
20	$x^{20} + x^{17} + 1$	1001000000000000000	0x90000	1,048,575
21	$x^{21} + x^{19} + 1$	10100000000000000000	0x140000	2,097,151
22	$x^{22} + x^{21} + 1$	11000000000000000000	0x300000	4,194,303
23	$x^{23} + x^{18} + 1$	100001000000000000000	0x420000	8,388,607
24	$x^{24} + x^{23} + x^{22} + x^{17} + 1$	1110000100000000000000	0xE10000	16,777,215

图 10: LFSR 多项式表

4.6 TextLCD 显示系统

4.6.1 队列设计

Queue 模块充当数据缓冲区。由于 TextLCD 显示需要一定宽度的音符数据流, 该模块将来自 ROM 的串行数据存入移位寄存器阵列。模块接收分频时钟 `clk_div`, 每当

```
// LFSR Algorithm
assign next_bit      = lfsr_reg[12]
                    ^ lfsr_reg[11]
                    ^ lfsr_reg[10]
                    ^ lfsr_reg[7]
                    ^ lfsr_reg[0];
```

图 11: LFSR 代码

时钟周期到来时，内部数据逐位右移，最低位被新的输入数据替换。队列最左侧的数据被作为“当前判定位”，同时送往显示模块和判定模块。

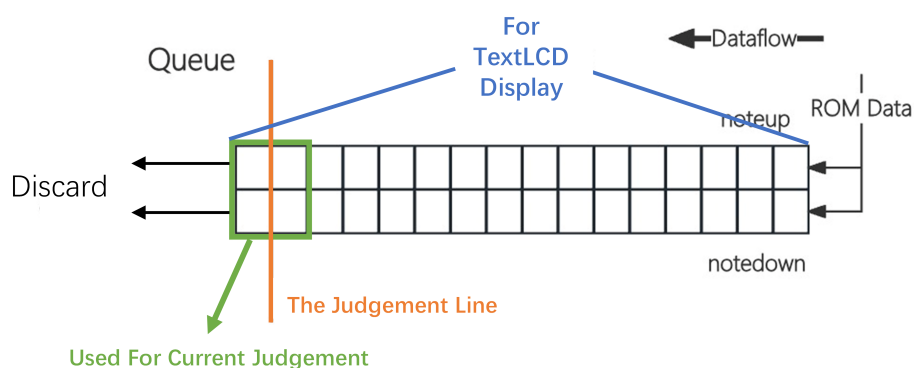


图 12: 队列模块设计原理

4.6.2 LCD 控制逻辑

TextLCD 模块负责生成驱动 LCD1602 所需的控制信号（如 RS, RW, EN）。由于 HD44780 控制器需要严格的初始化序列，我们设计了一个循环状态机来管理显示流程：空闲 → 清屏 → 设置显示模式 → 设置 DDRAM 地址 → 写入数据。模块将队列中的音符编码（0/1/2/3）映射为 ASCII 字符（空格、O、<、=），并通过设置 DDRAM 地址将字符写入屏幕的对应位置。

	显示位置	1	2	3	4	5	6	7		40
DDRAM	第一行	00H	01H	02H	03H	04H	05H	06H		27H
地 址	第二行	40H	41H	42H	43H	44H	45H	46H		67H

图 13: HD44780 DDRAM 地址映射表

4.6.3 双频率设计

在 TextLCD 模块中，我们引入了独立的缓冲时钟 `clk_buf`。这是因为游戏逻辑时钟 `clk_div` 的频率由音乐 BPM 决定，通常较低；直接用其驱动屏幕刷新会导致严重的闪烁和上下轨道显示不同步。通过引入较高频率的 `clk_buf` 专门负责 LCD 的指令写入与刷新，我们确保了即使在慢速歌曲中，屏幕显示依然流畅稳定。

4.7 七段数码管显示

DE2-115 开发板上的七段数码管由独立引脚控制，因此，我们使用组合逻辑电路将输入的 BCD 码或状态码译码为 7 段电平信号。模块 `TotalScore7Seg` 负责将累加器的数值转换为数字字形 (0-9)，而 `CurrentJudge7Seg` 则将判定结果映射为字符字形 (PF, GO, FL)，实现了判定结果的视觉反馈。

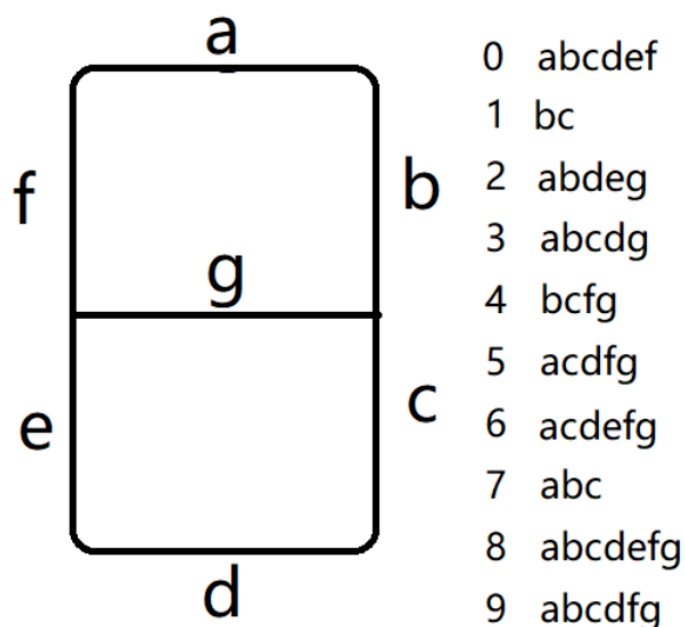


图 14: 七段数码管显示原理

4.8 判定逻辑

Judgement 模块是系统的核心，负责根据当前音符 (`cur_note`) 和按键输入做出评分。为了简化代码，我们定义了相关的宏来表示判定结果 (PERFECT, GOOD, MISS) 和音符类型。

4.8.1 硬件判定标准

对于 TAP 和 HOLD_START 音符，我们采用时间窗口判定法。判定窗口定义为两个 `clk_div` 周期。若按键上升沿落在窗口中间 $\pm 1/2$ 区域内，判定为 PERFECT；若在

```

`define PERFECT 2'b00
`define GOOD 2'b01
`define MISS 2'b10
`define NO_NOTE 2'b11

`define NOTHING 2'b00
`define TAP 2'b01
`define HOLD_START 2'b10
`define HOLD_MIDDLE 2'b11

`define PUSHED 1'b0
`define NOT_PUSHED 1'b1

```

图 15: 判定逻辑宏定义

窗口边缘，判定为 GOOD；若超出窗口未操作，判定为 MISS。对于 HOLD_MIDDLE 音符，由于没有设计 HOLD_END 事件，我们规定只需在前半个周期内保持按住即可判定为 PERFECT，后半周期不进行检测。这一设计结合了 count2 计数器来实现高精度的区间检测。

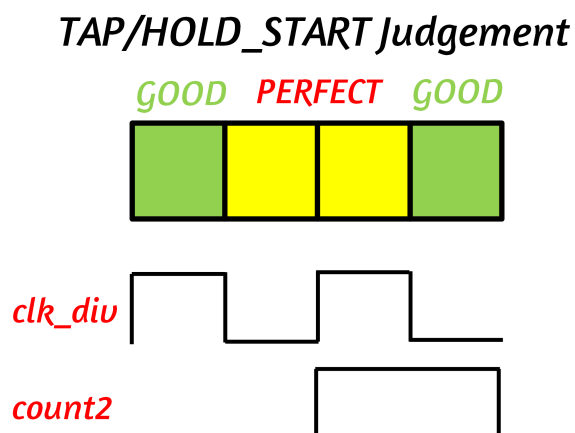


图 16: TAP 与 HOLD_START 判定时序图

4.8.2 判定信号锁定 (Signal Judged)

为了防止对同一个音符进行重复计分，我们引入了 *judged* 信号。该信号独立于状态机运行，其逻辑如下：当检测到有效判定（如 TAP 被击中）或时间窗口结束时，*judged* 信号置 1；当进入新的音符周期（即 *clk_div* 上升沿且 *count2* 下降沿）时，信号复位为 0。这种分离设计允许状态机专注于状态流转，而 *judged* 信号负责锁存结果。

HOLD_MIDDLE Judgement

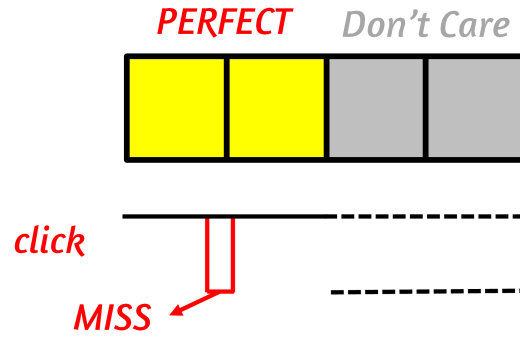


图 17: HOLD_MIDDLE 判定时序图

```

if(!rst_n) begin
    noteup_judged <= 1'b0;
end else begin
    if(cur_noteup == `NOTHING && cur_noteup_delay == `NOTHING && cur_noteup_delay2 == `NOTHING) begin
        noteup_judged <= 1'b0;
    end else if(clk_div_posedge && count2_negedge) begin
        noteup_judged <= 1'b0;
    end else if(clickup_posedge && (cur_noteup == `TAP || cur_noteup == `HOLD_START)) begin
        noteup_judged <= 1'b1;
    end else if(count2 == 1'b1 && cur_noteup == `HOLD_MIDDLE) begin
        noteup_judged <= 1'b1;
    end else if(!noteup_judged && cur_noteup == `HOLD_MIDDLE && clickup == `NOT_PUSHED && count2 == 1'b0) begin
        noteup_judged <= 1'b1;
    end else begin
        noteup_judged <= noteup_judged;
    end
end
end

```

图 18: Judged 信号逻辑代码

4.8.3 有限状态机设计

我们采用了单一 FSM 架构，包含四个状态：**NO_NOTE**, **GOOD**, **PERFECT**, **MISS**。状态机在 **clk** 时钟下运行，根据当前音符类型和按键输入实时更新状态。

默认情况下 FSM 处于 **NO_NOTE** 状态。当检测到 **TAP** 或 **HOLD_START** 音符时，FSM 预先进入 **GOOD** 状态以监听按键输入。若按键在窗口中心被按下，跳转至 **PERFECT**，否则保持或跳转至 **MISS**。



图 19: 判定逻辑的状态转移图

在调试过程中，我们发现了信号穿透的问题：**cur_note** 信号在时钟切换时会产生毛刺，导致 **HOLD_MIDDLE** 信号穿透到前一个周期造成误判。为了解决这个问题，我们在 FSM 中加入了一个强制跳转规则：在每个音符周期的中间点（即 **clk_div** 上升沿），强制状态机短暂跳转回 **NO_NOTE**。这一操作相当于一次“软复位”，有效切断了错误信号的传播路径，确保了逻辑的正确性。

4.8.4 Vivado 行为级仿真

鉴于判定逻辑的复杂性，我们在上板前进行了详尽的 Vivado 行为级仿真。仿真波形验证了五种关键场景：无操作导致的 **MISS**、精准点击导致的 **PERFECT**、长按时间不足导致的 **MISS** 等。仿真结果证明了逻辑设计的完备性。

```

if(count2 == 1'b1 || (count2_negedge && !clk_div_posedge)) begin
    cur_noteup <= left_noteup;
    cur_notedown <= left_notedown;
end else if(clk_div_posedge) begin
    cur_noteup <= `NOTHING;
    cur_notedown <= `NOTHING;
end else begin
    cur_noteup <= right_noteup;
    cur_notedown <= right_notedown;
end

```

图 20: 信号穿透问题的代码修正

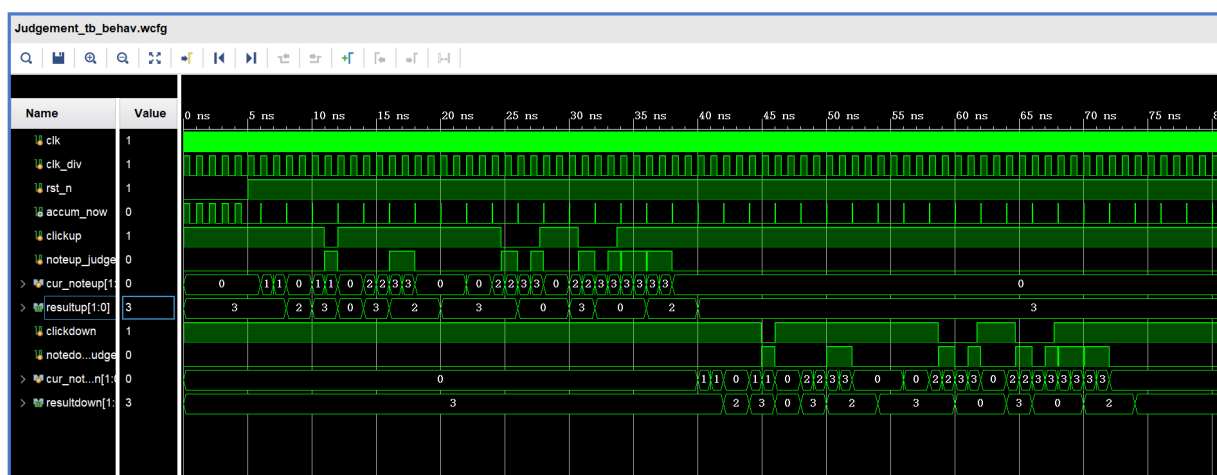


图 21: Judgement 模块的 Vivado 仿真波形

4.9 分数累加

分数累加模块负责游戏得分的实时计算与存储。为了解决 BCD 码运算的复杂性并确保系统的可维护性，我们在该模块中采用了层次化设计的策略，通过自底向上的方式构建运算单元。

4.9.1 核心逻辑与层次化构建

遵循层次化设计的思想，我们首先设计了最底层的基本单元——1 位 BCD 加法器 (bcd_adder)。该模块遵循标准的 BCD 加法修正规则：当两个 BCD 码相加的和超过 9 (1001) 或产生进位时，必须加 6 (0110) 进行修正。

基于这个基础单元，我们向上构建了更高层级的逻辑：

1. **级联扩展**：将 4 个 1 位 BCD 加法器级联，构建出支持 0-9999 范围的 4 位 BCD 加法器。
2. **模块封装**：最终将加法器封装进 Accumulator 模块，配合时序逻辑实现分数的持续累积。

这种设计不仅将复杂的 16 位运算分解为可管理的子任务，还隔离了不同层级的逻辑。当计分规则发生变化时，只需修改底层模块而无需重构整个系统。

```
wire [4:0] temp;
wire [4:0] temp_out;

assign temp      = a_extend + b_extend + cin_extend;
assign temp_out  = (temp > 5'd9) ? (temp + 5'd6) : temp;
assign cout      = temp_out[4];
assign s         = temp_out[3:0];
```

图 22: 1 位 BCD 加法器代码

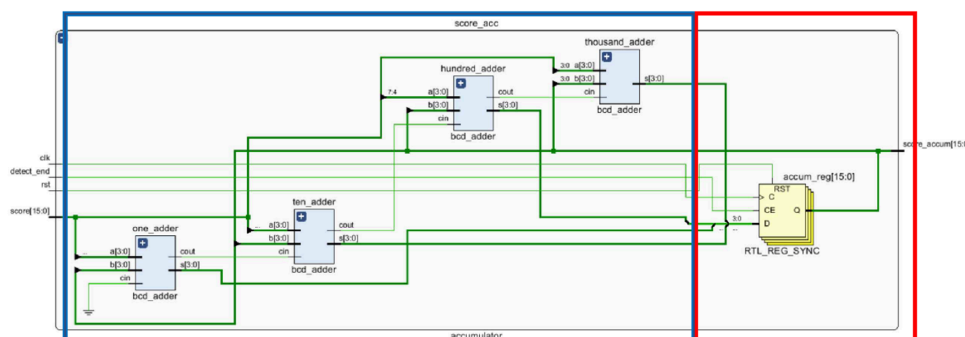


图 23: 分数累加器原理图

4.9.2 预加法优化

为了进一步优化累加器的端口设计，我们引入了 `ScoreConversion` 模块进行“预加法”处理。该模块在数据进入累加器之前，先将上轨道 (`score_up`) 和下轨道 (`score_down`) 的分数进行求和。

$$score = score_up + score_down \quad (2)$$

这种设计避免了在累加器内部设计复杂的三输入加法器，降低了单层逻辑的复杂度，同时也使得修改计分规则更加灵活。

5 讨论

5.1 设计技巧

- 避免生成锁存器 (Latches)：在组合逻辑中完整覆盖 `if-else` 或 `case` 分支。
- 优先使用 `assign` 语句代替 `always` 块中的组合逻辑。
- 规范命名（如 `rst_n` 表示低电平复位）和注释。
- 时序逻辑中使用非阻塞赋值 (`<=`)。

5.2 信号穿透

这是一个关键问题。`cur_note` 信号在切换时会出现毛刺 (Glitch)，导致 `HOLD_MIDDLE` 信号在时钟偏差下“穿透”到前一个周期，造成错误判定。**解决方案：**在判定区间的中间点，强制将状态置为 `NO_NOTE` 一次。这相当于一次软复位，阻断了错误信号的传播。

5.3 软硬件不一致性

Vivado 仿真完美，但上板出现问题（如 FSM 卡死在 `NO_NOTE`，或按键对 TAP 无响应）。由于无法定位根源，只能通过添加补丁代码（如强制状态跳转）来解决。

5.4 Quartus 调试

使用了 Signal Tap Logic Analyzer 进行片上调试。配置采样深度 (2K) 和触发条件 (Basic AND/OR)，观察实时波形以定位问题。

6 总结

本项目成功地在 DE2-115 上实现了一个基于 Muse Dash 的节奏游戏。在原有 Muse Dash on FPGA 音乐游戏实现的基础上，对“硬件写死、模式单一、交互割裂”的问题进行了系统性重构和扩展。通过引入以 Python 为核心的软件后端，将谱面生成、参数配置与分析逻辑从 Verilog 中解耦，构建了前端—后端—FPGA 协同的工程化流程。在此基础上，对谱面策略与交互模式进行抽象，支持多种生成路径与灵活切换，并通过可视化前端实现了配置、预览、下发与反馈的一体化闭环。最终形成了一个具备可配置性、可扩展性与基本用户体验的音乐游戏 FPGA 框架，实现了接近 MVP 的工程实现，也验证了软硬件协同在此类交互式 FPGA 应用中的可行性与扩展潜力。

7 未来展望

1. **消抖延迟优化：**目前的消抖模块引入了 $> 20\text{ms}$ 的延迟，对于判定区间仅 $\pm 50\text{ms}$ 的节奏游戏影响较大。未来需要在判定逻辑中补偿这一延迟。
2. **软硬件协同机制部署：**目前基于 UART 的音频同步方案仅完成了理论验证与架构设计。未来计划完成上位机与 FPGA 间的串口通信代码开发与板级调试，真正打通数据链路，实现“一键启动”的精确音画同步。